



Lecture 12: Conditional Rendering

INTERVIEW QUESTION

[> CORE](#)

How do you show, hide, or branch parts of the UI based on state?

You only want to show the login form when the user is signed out, and the dashboard when they're signed in. How do you conditionally render markup?

A static web page looks the same to every visitor. A web application, however, must adapt its interface based on the user's state, permissions, or actions.

The CSS Hiding Vulnerability

Imagine you are building a banking application. Users must log in before they can view their account balance. In older vanilla JavaScript patterns, developers would often render both the login form and the private dashboard into the HTML, and then use CSS (`display: none`) to hide the dashboard until the user authenticated.

This approach is catastrophic for security and performance. Even if the dashboard is visually hidden, a malicious user can simply open their browser's Developer Tools, inspect the DOM, and see the entire private dashboard HTML structure sitting right there. Furthermore, the browser is forced to download and parse heavy dashboard components that the user might never even see. You need a way to physically prevent HTML from existing in the document until a specific condition is met.

The `{#if}` Block

Svelte solves this using the `{#if}` logic block. It allows you to wrap portions of your markup in conditional statements.

```
App.svelte
1  <script>
2    let loggedIn = $state(false);
3  </script>
4
5  {#if loggedIn}
6    <h1>Welcome to your Dashboard!</h1>
7  {/if}
```

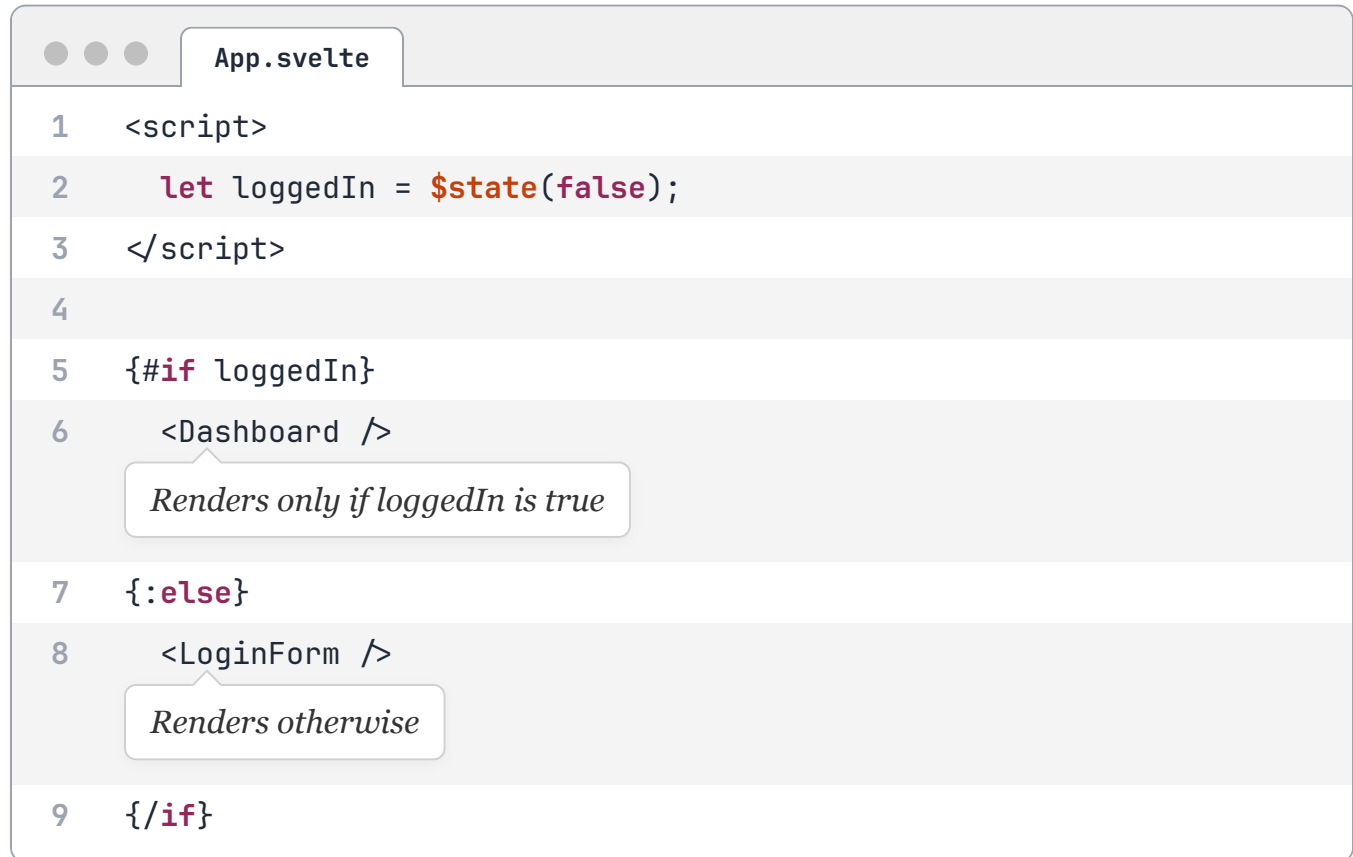
Unlike CSS toggles, Svelte's `{#if}` block mounts and unmounts elements. If `loggedIn` is false, the `<h1>` does not exist in the DOM at all. It is a completely impenetrable void. When `loggedIn` becomes true, Svelte instantly constructs the HTML and injects it into the page.

KEY TAKEAWAY

Logic blocks in Svelte start with a `#`, can have intermediate branching steps starting with a `:`, and always close with a `/`.

Branching with `{:else}`

Often, you don't just want to show or hide one thing; you want to toggle between two distinct interfaces. You can chain an `{:else}` block to handle the fallback state.



```
1  <script>
2    let loggedIn = $state(false);
3  </script>
4
5  {#if loggedIn}
6    <Dashboard />
7  {:else}
8    <LoginForm />
9  {/if}
```

Renders only if loggedIn is true

Renders otherwise

Complex Logic with {:else if}

For scenarios with more than two possible states, you can chain multiple conditions together using `{:else if}`. The first condition that evaluates to true will render its block, and Svelte will ignore the rest.



The screenshot shows a code editor window titled "TrafficLight.svelte". The code is as follows:

```
1  <script>
2    let status = $state('warning');
3  </script>
4
5  {#if status === 'danger'}
6    <p class="red">Stop immediately!</p>
7  {:else if status === 'warning'}
8    <p class="yellow">Proceed with caution.</p>
9  {:else}
10   <p class="green">All clear to go.</p>
11  {/if}
```

Summary

To conditionally render parts of your UI, Svelte uses the `{#if}` , `{:else if}` , and `{:else}` logic blocks. Crucially, these blocks do not merely visually hide elements using CSS; they physically construct and destroy DOM nodes based on reactive state. This ensures your application remains secure, lightweight, and perfectly synchronized with its underlying logic.

	CSS HIDING (Legacy)	SVELTE <code>{#if}</code> (Modern)
DOM Presence	Always in the DOM	Physically added/ removed
Security	Vulnerable to DOM inspection	Secure (HTML does not exist)
Syntax	<code><div class={isHidden ? 'hide' : ''}></code>	<code>{#if isVisible} <div> {/if}</code>